

Haru Health

계약 → 현장 결제 결과 기록 → 정산 → 출석으로 이어지는
멀티테넌트 체육관 운영 B2B SaaS 백엔드

Java 21

Spring Boot 3.5

MyBatis

PostgreSQL

JWT

SSE

React 19

송태현 · Backend Developer

uiop7000@naver.com · github.com/Taehyun-0502/health_Project

4 인 팀 · 2026.06.29 – 07.28
요구사항 117 건 · 테이블 27 개 · 미배포 (로컬 검증)

목차 — 담당 범위 → 흐름 · 아키텍처 → 사례 5 건 → 검증 · 한계

제목이나 페이지를 누르면 이동하고, 모든 장 오른쪽 아래의 '🏠 목차' 로 돌아옵니다.

01

[PROJECT — 담당 범위 · 업무 흐름 · 아키텍처 · 데이터](#)

4 인 팀 · 27 테이블 · payment / settle / item / checkInout / alarm

[03 - 06 →](#)

02

[CASE 01-02 — 결제 트랜잭션 · 출석 PT 원장](#)

행위자 / 대상 분리 · 조건부 UPDATE · 키오스크 레이트 리미터

[07 - 08 →](#)

03

[CASE 03-05 — 정산 배치 · LATERAL SQL · 알림 예외 처리](#)

advisory lock · row fan-out · sendAlarmSafely · SSE 티켓

[09 - 11 →](#)

04

[VALIDATION — 확인한 것 · 한계 · 다음 \(→ 땡땡땡에서 실행\)](#)

미배포 · permitAll · 실제 DB 롤백 · 락 검증 미흡

[12 →](#)

시간이 3 분뿐이라면

이 네 장만 보셔도 됩니다. 제목을 누르면 이동합니다.

03

[담당 범위 한 장](#)

팀 전체 vs 내 것

05

[아키텍처](#)

React · Spring · PostgreSQL · FastAPI

07

[결제 트랜잭션](#)

6 단계 원자성

12

[검증 · 한계](#)

검증 범위와 남은 과제

서비스 전체 범위와 개인 담당을 구분했습니다 — 정산 · 결제 · 출석 · 알림

4 인 팀에서 payment · settle · item · checkInout · alarm 의 Service·Mapper 계층을 담당했습니다 . 개인 기여는 Git 이력과 코드로 확인되는 범위만 적었습니다 .

팀 전체 구현 범위

- 관리자 → 사장님 → 트레이너 → 회원 4 계층 역할별 대시보드
- 전자계약 · 서명 (팀원)
- 현장 결제 결과 기록 (외부 PG 미연동) · 매출 · 정산 · 물품 · CSV
- 출석 키오스크 · PT 관리 · 일정
- 이탈 예측 XGBoost + SHAP (팀원 , FastAPI)
- AI 운영 비서 Tool Use (팀원)
- 실시간 알림 (SSE)

1 결제 · 쿠폰

현장 결제 기록 (외부 PG 미연동) · 6 단계 단일 트랜잭션 · 행위자와 기록 대상 분리

2 정산 · 물품

월 정산 배치 (advisory lock) · 지급 후 확정값 보존 · 물품 구매 → 지출 자동 연결 · CSV

3 출석 · PT 원장

무로그인 키오스크 (레이트 리미터) · 트레이너 확인 시점 차감 · h_pt_manage 독자 원장

4 알림 · SSE

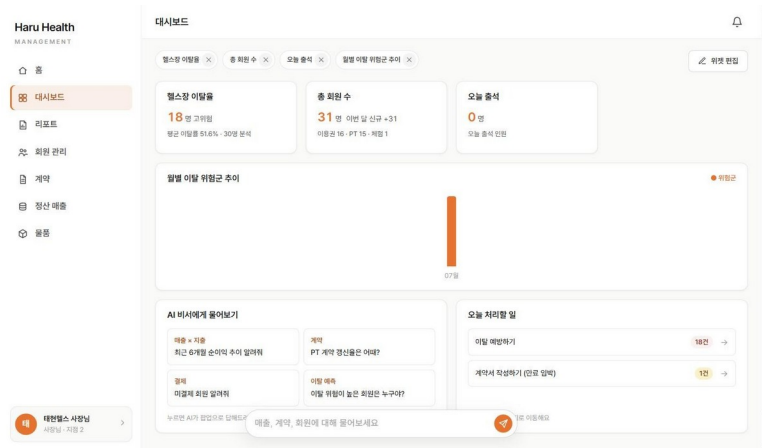
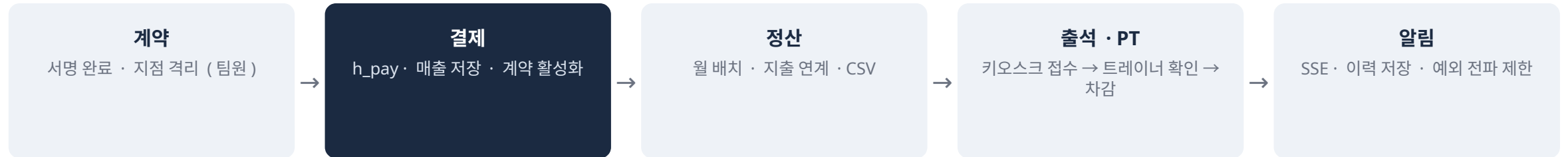
h_alarm 이력 + SSE 실시간 발송 공용 메서드 · 1 회용 구독 티켓 · sendAlarmSafely 예외 전파 제한

검증 근거

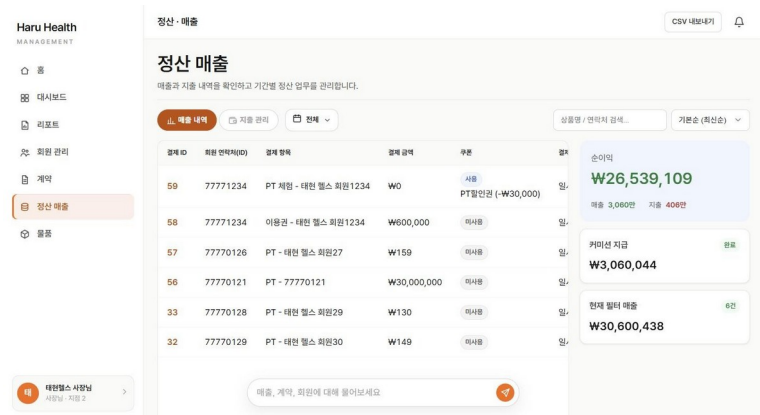
54 개 이상 PR 기반 협업 · 결제 · 정산 · 물품 쿼리와 서비스 흐름 문서화 · docs/ 내가 _ 작성한 _ 쿼리 _ 메서드 _ 설명 .md

계약 → 현장 결제 결과 기록 → 정산 → 출석 — 사장님 기록부터 트레이너 확인까지

현장 결제 결과는 사장님이 기록하고, 정산은 월 배치가 만들며, PT 차감은 트레이너가 확인하는 순간 일어납니다.



사장님 대시보드 (팀 화면)



정산 화면 · 지급 처리 (담당)



출석 키오스크 (담당)

화면 : 로컬 시드 데이터로 구동한 실제 React 화면 · 배포 환경 · 실사용 수치는 없어 제외했습니다.

React → Spring(MyBatis) → PostgreSQL — 내 담당은 결제 · 정산 · 출석 · 알림 서비스 계층

단일 DB · 단일 스키마 row-level 멀티테넌트 · 화면이 보낸 지점 번호는 믿지 않고 JWT 의 username 으로 회원을 재조회해 gym_id 를 강제합니다 .



27 개 테이블을 6 개 업무 축으로 — 인증된 회원의 gym_id 로 데이터 격리

gym_id 조건으로 지점을 구분합니다. 타 도메인 테이블은 SELECT 만 하고 필요한 상태는 내 원장에 기록합니다.

멀티테넌트 격리 원칙

JWT 의 username 으로 회원을 재조회해 gym_id 를 적용하고 소속이 없으면 -1 로 차단. 쿼리별 격리 검증과 PostgreSQL RLS 도입은 추가 과제

```
-- 결제 · 정산 쿼리의 테넌트 격리
WHERE gym_id = (SELECT gym_id FROM h_member
                WHERE username = #{username})
```

MEMBER & GYM

h_member · h_gym · h_bank

사용자 · 역할 · 지점 원천

CONTRACT (팀원)

h_contract · h_contract_data · h_sign

전자서명 · 상태 · 기간

PAYMENT & SETTLE

h_pay · h_payment · h_settlement · h_expense

결제 · 매출 · 지출 · 커미션

ATTENDANCE & PT

h_check_inout · h_pt_manage · h_pt_schedule

접수 · 확인 · 차감 원장 · 일정

CHURN & SURVEY (팀원)

h_model_data · h_survey · h_churn_result

피처 · 설문 · 확률 · SHAP 원인

ALARM

h_alarm

이력 · 읽음 · 링크 · 카테고리

■ 강조 = 내가 설계 · 소유한 축 · 타 도메인 테이블은 SELECT 만 하고 필요한 상태는 내 원장에 기록 (' 쓰기는 소유 도메인만 , 타 도메인은 읽기 + 파생 테이블 ')

현장 결제 결과 기록 6 단계를 한 트랜잭션으로

현장 결제 결과 · 매출 기록과 계약 활성화 서비스 호출을 한 트랜잭션으로 묶었습니다. 실제 DB 롤백은 추가 검증 과제입니다.

PayService.checkout — 요약 의사코드 (실제 코드 : payment 패키지)

```
@Transactional
public void checkout(Long dataId, String owner, Long couponId) {
    // ① 사장님이 발행 · 서명 완료 · 미결제 계약인지
    Contract c = findPayableContract(dataId, owner); // sender = 로그인 사장님
    // ② 쿠폰 : 소유자 ( 계약 수신자 ) · 미사용 · 만료 · 지점 · 카테고리
    int amount = applyCoupon(c.getReceiverId(), couponId, c);
    if (amount == 0) installment = 0; // 0 원이면 할부 강제 해제
    insertPay(c, amount); // ③ h_pay
    markCouponUsed(couponId); // ④
    insertRevenue(c, amount); // ⑤ h_payment ( 매출 )
    if (!activateContract(c)) throw new IllegalStateException(); // ⑥
}
```

서비스 분기 구현 · 실제 DB 롤백 · 락 검증 필요

- 코드 분기 : 타 지점 회원 매출 등록 거부
- 타 지점 매출 삭제 · 재계산 거부

1 행위자와 기록 대상을 분리

처음엔 회원이 결제 결과를 직접 기록한다고 가정했습니다. 실제 흐름은 카운터에서 사장님이 현장 결제 결과를 기록하므로 sender 와 기록 대상 회원을 분리했습니다.

2 스키마가 두 번 바뀌어도 견디게

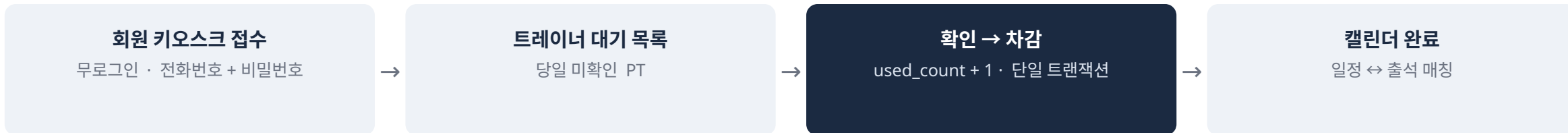
쿠폰이 1 테이블 → 2 테이블 (발급 · 템플릿) 로 재설계되고 컬럼도 교체됨. 헬스 / PT 이분법 하드코딩을 카테고리 → 규칙 매핑으로 바꿔 체험권은 case 하나만 추가

알고 있는 한계

- 동시 결제 검사를 둘 다 통과할 수 있음. 계약별 복수 결제의 허용 여부를 정의한 뒤 유니크 제약 · 계약 행 잠금 결정
- 외부 PG 연동 없음 — 현장 결제 기록 모델

PT 원장 — 트레이너 확인 시 조건부 UPDATE 로 차감

팀원이 담당한 계약 테이블은 조회만 하고, 출석 도메인의 원장 h_pt_manage 에 사용량을 기록합니다. DB 의 원자적 UPDATE 로 차감합니다.



```

-- 잔여 횟수는 원본과 원장에서 파생 ( 별도 저장 안 함 )
remaining = contract.quantity - pt.used_count

-- 조건부 UPDATE + 영향 행 수 검사 : 잔여 없으면 0 건 → 예외 → 롤백
UPDATE h_pt_manage
  SET used_count = used_count + 1,
      completed = (used_count + 1 >= total_count)
 WHERE data_id = :dataId AND used_count < total_count;
  
```

왜 별도 원장인가

계약 원본과 도메인 소유권을 지키고, completed=true 행은 팀원이 담당한 계약 서비스가 소비해 종료 처리하도록 인터페이스를 나눴습니다.

1 무인증 키오스크의 브루트포스 방어

AttendanceRateLimiter — 계정 축 (5 회 실패 → 1 분 잠금) + IP 축 (20 회 → 5 분).
계정 기준만으로는 password spraying 을 못 막아 두 키를 씀. 잠금은 429 로 구분 응답

2 초과 차감 제한과 중복 처리는 별도 검증

조건부 UPDATE 는 잔여량 초과를 제한. 동일 출석의 중복 차감은 확인 상태 전이와 함께 검증 필요. 하루 1 회 출석도 check-then-act 라 날짜 · 회원 유니크 제약 검토

월 정산 중복 생성을 막는 키 잠금 — 지급 완료 금액은 보존

매월 1 일 00:00 전월 매출 기준 커미션 생성 . 아직 INSERT 되지 않은 행은 SELECT FOR UPDATE 로 잠글 수 없어 지점별 키 잠금을 썼습니다 .

SettleMapper — 요약 (실제 SQL: settle 매퍼 XML)

```
-- 지점별 '키 잠금' — 트랜잭션 종료 시 자동 해제
SELECT pg_advisory_xact_lock("#{gymId}");
-- 존재 확인 후 0 건일 때만 INSERT → 재실행 · 동시 실행에도 정산 1 건
SELECT COUNT(*) FROM h_settlement
  WHERE gym_id = #{gymId} AND settle_month = #{month};

-- 지급 처리 : '미지급' 일 때만 갱신, 영향 행 != 1 이면 '이미 지급됨'
UPDATE h_settlement SET paid = true, paid_at = now()
  WHERE settlement_id = #{id} AND paid = false;
```

지급 금액은 요청 body 를 버리고 DB 값으로

- 클라이언트가 금액을 조작할 수 없도록 정산 레코드 값을 기준으로 처리
- 코드 분기: 중복 지급 거부 · 당월 조기 마감 방지

1 매출 삭제 시 재계산 규칙

그 달 정산 없음 → 무시 · 미지급 → 남은 매출 × 수수료율로 재계산 · 지급 완료 → 자동
감액 금지, 관리자 확인 알림만

2 물품 등록 → 지출 자동 연결

구매 (양수) 면 같은 트랜잭션으로 expenseAdd. useGeneratedKeys 로 받은 item_id 를
origin_item_id 에 저장해 추적선 유지 . 정산에 반영된 지출 삭제는 FK 500 대신 사전 체크
+ 400

3 한계

단일 인스턴스 전제 · 미배포 . 잠금 · 존재 확인의 동시 실행은 실제 DB 검증 필요 . 스케줄러
4 개의 zone 누락은 수정 과제이며 , UTC 서버에서는 실행 시각이 달라짐

매출 목록 페이지가 깨졌다 — 조인의 row fan-out 을 LATERAL ... LIMIT 1 로

같은 계약에 결제 행이 2 건이면 매출 1 건이 여러 행으로 늘어나는데, 건수 쿼리는 그 조인을 하지 않아 목록과 총 건수가 어긋났습니다.

BEFORE — 요약

```
-- BEFORE: 결제 2 건 → 매출 1 건이 2 행으로 (row fan-out)
SELECT pm.*, p.pay_method
  FROM h_payment pm
  JOIN h_pay p ON p.data_id = pm.data_id
 WHERE pm.gym_id = #{gymId}
 ORDER BY pm.payment_id DESC LIMIT 10 OFFSET 0;
```

AFTER — 요약 (실제 SQL: payment 매퍼 XML)

```
-- AFTER: 바깥 행마다 최신 결제 1 건만 붙인다
SELECT pm.*, p.pay_method
  FROM h_payment pm
  LEFT JOIN LATERAL (
    SELECT pay_method FROM h_pay
    WHERE data_id = pm.data_id
    ORDER BY p_id DESC LIMIT 1) p ON true
 WHERE pm.gym_id = #{gymId} ... -- '일반 조인으로 바꾸지 말 것' 주석
```

조회 결과 — 목록과 COUNT 일치. 복수 결제의 허용 이력·오류 중복 구분은 추가 정의가 필요하며, 그 기준에 맞춰 조회·제약을 결정합니다.

1 MyBatis 를 고른 이유

대시보드·정산·출석 집계처럼 조인·집계·동적 정렬이 많은 화면 → SQL 직접 제어.
PostgreSQL 전용 (LATERAL·DISTINCT ON·FILTER·ON CONFLICT·advisory lock) 활용

2 목록·건수·CSV 세 쿼리의 조건 일치

같은 WHERE 를 복붙하던 것을 <sql> 조각 + <include> 로 모아 구조적으로 어긋나지 않게.
정렬은 <choose> 로 허용 값만, 동점 순서 안정화를 위해 PK 를 2 차 정렬 키로

3 파라미터 바인딩·응답 크기 제한

검색어는 #{ } 바인딩, \${ } 미사용. Pager 의 pageSize 상한 100 으로 요청당 조회량 제한.
반복 조회까지 차단하는 기능은 아님

알림 예외 전파를 제한한 처리 — try/catch 의 효과와 트랜잭션 한계

정산·매출 처리 중 알림 호출의 예외를 잡도록 변경했습니다. 호출부 예외 전파는 제한하지만, 알림 DB 오류까지 핵심 트랜잭션에서 격리한 것은 아닙니다.

sendAlarmSafely — 요약

```
// 호출부 예외 전파 제한 — DB 실패의 롤백 격리는 별도 검증
private void sendAlarmSafely(String receiver, String msg, String link, String
cat) {
    try {
        alarmService.sendAlarm(receiver, null, msg, link, cat); // 이력 저장 + SSE
    } catch (Exception e) {
        log.warn("알림 발송 실패 (핵심 처리는 유지) receiver={}", receiver, e);
    }
}
// 기존 로그의 '핵심 처리는 유지' 문구는 커밋 증거가 아님
```

try/catch 의 적용 범위

- SSE 전송 실패와 알림 이력 DB 실패를 구분해야 합니다. DB 오류나 rollback-only 상태는 catch 만으로 복구되지 않을 수 있어 실제 DB 검증이 필요합니다.
- 댕댕에서는 커밋 후 방송에 AFTER_COMMIT 을 적용했습니다. 비동기 실행·전달 보장은 별도입니다.

1 SSE 구독 보안 — 1 회용 티켓

- EventSource 는 Authorization 헤더를 못 보냄 → Bearer 토큰으로 티켓 발급 → 티켓으로 /alarm/subscribe 연결
- SecureRandom 32 바이트 · TTL 60 초 · 조회 즉시 remove(). 토큰을 쿼리스트링에 넣어 로그에 남기는 것을 피함

2 함께 잡은 버그 2 건

- gymOwnernum 을 사장님 전화번호로 오해해 알림 수신자로 썼다가 FK 위반 — 실제로 사업자등록번호 · gym_id + role=OWNER 조회로 교체 ('필드 이름을 믿지 말고 데이터를 확인한다')
- 백엔드 미기동 시 EventSource 무한 재접속 폭주 → onerror 에서 close 가드

무엇을 확인했고, 무엇이 아직 부족한지

코드에 구현한 처리와 실제 DB 로 검증할 과제를 구분했습니다.

● 코드 구현 범위

- 타 지점 접근 거부 분기
- 지급 금액 · 중복 지급 · 조기 마감 분기
- 현장 결제 결과 · 매출 기록 트랜잭션 경계
- PT 차감 조건부 UPDATE
- 알림 catch · SSE 구독 티켓
- 실제 DB 기반 롤백 · 락 · 동시성 시나리오 검증 미완료

● 현재 한계

- 미배포 — 운영 환경 성능 측정 없음
- 전역 permitAll + 컨트롤러별 수동 JWT 검증
- 리프레시 토큰 회전 없음 (유효기간 7 일)
- 실제 DB 롤백 · 락 · 동시성 검증 미흡
- 글로벌 예외 핸들러 없음 · -2 같은 반환 코드
- 동시 결제 · 중복 출석 · 알림 DB 실패 · 타임존 과제

● 후속 적용 · 남은 검증

- 땡땡땡 적용 : JWT 필터 · @AuthenticationPrincipal
- 땡땡땡 적용 : ErrorCode · 글로벌 예외 처리
- 땡땡땡 적용 : AFTER_COMMIT 방송
- 우선 : 실제 DB 롤백 · PT 중복 차감 · 정산 잠금 검증
- 다음 : 결제 이력 규칙 · 제약 결정, 출석 중복 · zone 수정

● TAKEAWAY

함께 성공해야 할 일과 실패해도 되는 일을 나누고, 남의 도메인은 읽고 내 원장에 기록했습니다

1

읽고 - 비교하고 - 쓰지 않는다

조건부 UPDATE 의 영향 행 수로 판정 · advisory lock · 지급 금액은 DB 값

2

클라이언트가 보낸 식별자를 믿지 않는다

JWT 의 username 으로 gym_id 조회 · 행위자 (사장님) 와 기록 대상 (회원) 분리

3

쓰기는 소유 도메인만 , 타 도메인은 읽기 + 파생 테이블

h_pt_manage 원장 · sendAlarmSafely · 다음 프로젝트의 설계 기준

송태현 · Backend Developer

uiop7000@naver.com · github.com/Taehyun-0502/health_Project